

Walidacja danych marketingowych względem materiałów źródłowych z użyciem LLM

Podsumowanie dla kierownictwa

Najbardziej praktycznym i najbezpieczniejszym wzorcem nie jest pojedynczy „LLM-judge”, który czyta cały rekord i wydaje wyrok, lecz pipeline etapowy: **rozbicie treści na atomowe claimy, wyszukanie właściwych fragmentów dowodowych, weryfikacja każdego claimu w formacie ściśle ustrukturyzowanym oraz selektywna automatyzacja z eskalacją przypadków niejednoznacznych do człowieka**. Taki układ jest zgodny z kierunkiem badań nad claim extraction i evidence-grounded verification, a jednocześnie ogranicza znane słabości LLM-as-a-judge, zwłaszcza bias pozycyjny i podatność na powierzchowne heurystyki. Badania nad jakościowym kodowaniem pokazują, że LLM potrafi zbliżyć się do ręcznego kodowania, ale badania nad span annotation pokazują też tylko umiarkowaną zgodność z ludźmi, więc pełna automatyzacja bez warstwy niepewności i adjudykacji jest metodologicznie zbyt ryzykowna. ¹

Jeżeli priorytetem jest **minimalizacja false positives**, system powinien być „konserwatywny”: rekord dostaje automatyczny status negatywny lub „do poprawy” tylko wtedy, gdy spełnione są równocześnie cztery warunki: claim jest dobrze zdekomponowany; retrieval dostarczył właściwe fragmenty źródłowe; co najmniej dwa niezależne przebiegi weryfikacji zgadzają się co do sprzeczności albo braku substancji; a pewność po kalibracji przekracza ustalony próg. Wszystko, co nie spełnia tych warunków, powinno wpadać do koszyka **AMBIGUOUS / HUMAN REVIEW**, a nie do koszyka „błąd”. To podejście jest spójne z obserwacją, że LLM-y mają skłonność do zgadywania, gdy ewaluacja karze niepewność, oraz że mechanizmy abstention mogą ograniczać halucynacje przy niewielkim koszcie obliczeniowym. ²

Jeżeli projekt ma też wymiar compliance w Polsce i UE, etykiety muszą rozróżniać **fałsz, sprzeczność z materiałem, utratę zastrzeżenia, pominięcie informacji istotnej, brak substancji i niejednoznaczność**. W prawie unijnym praktyka jest myląca nie tylko wtedy, gdy zawiera informację fałszywą, ale także wtedy, gdy przez **overall presentation** lub pominięcie wprowadza przeciętnego konsumenta w błąd; podobną, case-by-case logikę oceny prezentacji i grupy odbiorców stosuje UOKiK. To oznacza, że walidacja marketingu nie może być redukowana do samego „string match”. ³

Punkt startowy, który zwykle najlepiej łączy jakość, koszt i wdrażalność, wygląda następująco: **pierwszy filtr tani lub lokalny model, drugi filtr mocniejszy model z JSON Schema, próg auto-flag ustawiony na wysoką precyzję, obowiązkowy human review dla szarej strefy oraz dla wysokiego risk-tier, a cały system stroi się na złotym zbiorze zrobionym przez dwóch ludzkich koderów i adjudykatora**. Dokumentacje OpenAI, Anthropic i Google jednoznacznie wspierają Structured Outputs/strict schemas, a literatura ewaluacyjna pokazuje, że testy powinny być wielowymiarowe, task-specific i oparte na rzeczywistym rozkładzie przypadków, nie na jednym syntetycznym benchmarku. ⁴

Warstwa	Rekomendacja robocza	Dlaczego to obniża false positives
Jednostka oceny	atomowy claim, nie cały rekord	model ocenia jedną tezę na raz, a nie miesza kilku wątków

Warstwa	Rekomendacja robocza	Dlaczego to obniża false positives
Źródła	wersjonowane dokumenty + evidence spans	ogranicza „zgadywanie” poza materiałem bazowym
Orzekanie	ściśle zdefiniowane etykiety + JSON Schema	redukuje błędy formatu i ułatwia audyt
Automatyzacja	selektywna, z abstention	przypadki niepewne nie stają się automatycznie błędami
Jakość	dwie niezależne ścieżki orzekania dla auto-flag	zmniejsza wpływ pojedynczego biasu modelu lub promptu
Benchmark	dwa ludzkie koderki/koderzy + adjudkacja	daje realny odpowiednik „human qualitative coders”

To zestawienie syntetyzuje wyniki badań nad claim extraction, retrieval-grounded verification, biasami LLM-as-a-judge, selective prediction oraz oficjalne możliwości structured outputs u głównych dostawców. ⁵

Kontekst, cele i założenia

Celem operacyjnym powinno być nie „zastąpienie ludzi LLM-em”, lecz **zbudowanie systemu wspomaganej walidacji**, który zachowuje się możliwie podobnie do dobrego koderka jakościowego: pracuje według codebooka, odwołuje się do wskazanych materiałów źródłowych, ujawnia dowody, konsekwentnie rozdziela niezgodność od niepewności oraz zostawia ślad audytowy. Takie ujęcie jest bliższe temu, jak opisują zadanie zarówno metodologia jakościowa, jak i współczesne praktyki budowy ewaluacji LLM. ⁶

Zakres, który da się wdrożyć bez nadmiernego ryzyka, obejmuje przede wszystkim: walidację tekstowych outputów marketingowych względem **approved source of truth**; wieloetykietową klasyfikację zgodności claimów; ekstrakcję wymaganych evidence spans; szacowanie niepewności; triage przypadków do człowieka; i stałą rekalkulację na nowych danych. Rozszerzanie od razu na całkowicie otwarty web, subiektywne oceny kreatywne lub rozproszone materiały multimodalne bez uporządkowanego repozytorium źródeł znacząco podnosi ryzyko błędów i powinno być drugim etapem, nie punktem startowym. ⁷

Najważniejsze założenia, których użytkownik nie doprecyzował, trzeba jawnie zamrozić w dokumencie projektowym przed uruchomieniem benchmarku:

Obszar	Domyślne założenie robocze	Jeżeli założenie jest fałszywe
Języki	pl-PL i en-US/en-GB są głównymi językami	trzeba dodać osobne slice’y ewaluacyjne i examples dla kolejnych języków
Modalność	rdzeń systemu jest tekstowy; obraz/PDF traktowane jako źródła do ekstrakcji tekstu lub multimodalnej weryfikacji tylko tam, gdzie to konieczne	potrzebne osobne benchmarki vision/OCR

Obszar	Domyślne założenie robocze	Jeżeli założenie jest fałszywe
Źródło prawdy	materiały zatwierdzone przez firmę są jedynym autorytatywnym ground truth	jeśli dopuszczony jest web lub know-how ekspercki, trzeba dodać warstwę source ranking i conflict handling
Pozytywna klasa	„problem” = sprzeczność, brak substancji lub utrata kwalifikatora wobec materiału źródłowego	jeśli „unsupported” nie ma być błędem, etykiety i progi muszą się zmienić
Ryzyko	wyższy próg ostrożności dla numerów, cen, comparatives, claims regulowanych i ofert czasowych	bez risk-tier system będzie zbyt jednolity i albo za drogi, albo zbyt liberalny

To, że założeń nie wolno pozostawić w domyśle, wynika również z praktyk ewaluacyjnych: success criteria muszą być specyficzne, mierzalne, osiągalne i ściśle związane z realnym taskiem. ⁸

Dla tego typu zadania proponuję sześć etykiet roboczych na poziomie claimu: **SUPPORTED, CONTRADICTED, NOT SUBSTANTIATED, QUALIFIER LOSS, AMBIGUOUS, OUT_OF_SCOPE**. To rozróżnienie jest praktyczne z dwóch powodów. Po pierwsze, publiczne benchmarki claim verification rozróżniają zwykle wsparcie, refutację i brak wystarczającego dowodu, co metodologicznie pasuje do tego problemu. Po drugie, w marketingu duża część kosztownych false positives bierze się nie z jawnej sprzeczności, tylko z wrzucenia „braku jednoznacznego dowodu” do worka „błąd”. ⁹

Dane, formaty i preprocessing

Warunkiem powodzenia jest **dobrze modelowanie danych**, nie tylko dobry model. Zewnętrznie zadanie przypomina document-level NLI i claim verification: mamy hipotezę marketingową, dokument lub zbiór dokumentów źródłowych oraz potrzebę zwrócenia werdyktu wraz z uzasadniającymi fragmentami. Publiczne zbiory takie jak DocNLI, ContractNLI i AVeriTeC dowodzą, że to nie jest zwykła klasyfikacja tekstu, tylko problem wnioskowania nad dokumentem i dowodem. ¹⁰

Minimalny format danych wejściowych powinien zawierać pięć bytów: rekord marketingowy, dokument źródłowy, evidence span, codebook oraz metadane audytowe. W praktyce najlepiej utrzymywać te byty osobno, z kluczami i wersjami, zamiast w jednym płaskim CSV, ponieważ później trzeba liczyć zgodność na różnych poziomach: claim, span, rekord, kampania, produkt i wersja materiału. AVeriTeC pokazuje, że jawna reprezentacja pytań, odpowiedzi, źródeł oraz uzasadnienia bardzo ułatwia późniejszą ocenę systemu. ¹¹

Byt	Pola wymagane	Uwagi praktyczne
Rekord marketingowy	<code>record_id</code> , <code>campaign_id</code> , <code>channel</code> , <code>lang</code> , <code>candidate_text</code> , <code>created_at</code> , <code>risk_tier</code>	<code>risk_tier</code> steruje programami i HITL
Dokument źródłowy	<code>doc_id</code> , <code>version</code> , <code>authority_rank</code> , <code>doc_type</code> , <code>language</code> , <code>canonical_text</code>	<code>version</code> jest krytyczne przy treściach czasowych
Segment / evidence span	<code>span_id</code> , <code>doc_id</code> , <code>start_char</code> , <code>end_char</code> , <code>section_title</code> , <code>span_text</code>	potrzebne do audytu i oceny retrieval

Byt	Pola wymagane	Uwagi praktyczne
Codebook	label_definition, inclusion_rules, exclusion_rules, examples, policy_overrides	bez codebooka LLM nie „emuluje kodera”, tylko improwizuje
Złota adnotacja	claim_id, gold_label, gold_spans, coder_a, coder_b, adjudicator	potrzebne do kappa/alpha i analizy błędów

Proponowany schemat rekordu może wyglądać tak:

```
{
  "record_id": "mk_000481",
  "campaign_id": "camp_q3_pl_12",
  "channel": "email",
  "lang": "pl-PL",
  "risk_tier": "high",
  "candidate_text": "Nasza formuła usuwa 99% przebarwień po 7 dniach.",
  "ground_truth_docs": [
    {
      "doc_id": "doc_claims_2026_02",
      "version": "2026-02-14",
      "authority_rank": 1,
      "doc_type": "pdf",
      "canonical_text": "...",
      "spans": [
        {
          "span_id": "sp_188",
          "start_char": 12450,
          "end_char": 12710,
          "section_title": "Claims substantiation",
          "span_text": "..."
        }
      ]
    }
  ],
  "codebook_id": "claims_v1_3"
}
```

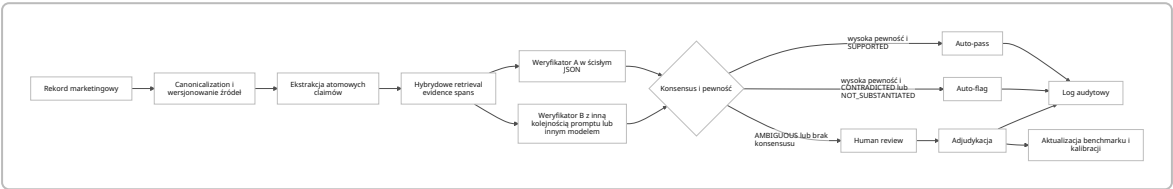
Ten format jest syntezą praktyk z benchmarków document-grounded oraz potrzeb produkcyjnych związanych z wersjonowaniem, audytem i adjudykacją. ¹²

Preprocessing powinien być osobnym, mierzalnym pipeline’em. Najpierw trzeba wykonać **canonicalization**: deduplikację dokumentów, normalizację kodowania, usunięcie boilerplate, OCR tylko tam, gdzie parser PDF lub ekstrakcja tekstu nie wystarcza, oraz wersjonowanie dokumentów. Potem następuje **segmentacja**: dzielenie źródeł na sensowne jednostki do retrieval, z offsetami znakowymi. Następnie **normalizacja semantyczna**: ujednolicenie liczb, jednostek miary, walut, dat i nazw własnych, bo właśnie tam najczęściej rodzą się kosztowne pozorne sprzeczności. Dopiero po tym warto uruchamiać LLM do ekstrakcji claimów. ¹³

Warto też rozdzielić dwa rodzaje przygotowania danych, które w praktyce często są mieszane. Pierwszy to **preprocessing produkcyjny** dla wszystkich rekordów. Drugi to **gold-set curation** dla benchmarku: dual coding przez ludzi, adjudykacja, zapis evidence spans i odnotowanie przyczyn niezgodności. Badania nad jakościowym kodowaniem z LLM i nad LLM-as-annotator sugerują, że to właśnie jakość codebooka i przykładowych adnotacji w największym stopniu decyduje o tym, czy model faktycznie zachowuje się jak koder, czy tylko generuje plausibly-sounding verdicts. 14

Architektura walidacji i opcje modeli

Pojedyncza ocena typu „czy ten output jest zgodny z materiałami?” jest kusząca, ale metodologicznie słaba. Literatura o LLM-as-a-judge pokazuje, że taka ocena jest podatna na bias pozycyjny, a badania nad fine-tuned judge models pokazują, że nawet wyspecjalizowane judge'e nie są ogólnym substytutem silnego modelu frontierowego ani człowieka. Dlatego sensowniejsza jest architektura, w której model wielokrotnie rozwiązuje mniejsze, lepiej zdefiniowane zadania. 15



Taki przepływ odpowiada temu, co wiemy z claim verification: realistyczne pipeline’y korzystają z dekompozycji claimu, retrieval i osobnej warstwy weryfikacji, a nie z jednego kroku. Dobrym kompromisem jest retrieval hybrydowy — indeks leksykalny plus embeddingowy — a następnie weryfikacja nad kilkoma najlepszymi, ale krótkimi spanami, nie nad całym korpusem. To zwykle poprawia zarówno trafność, jak i audytowalność. 16

Porównując podejścia, różnica praktyczna wygląda następująco:

Metoda	Co robi	Zalety	Główne wady	Ryzyko false positive
Reguły i regex	sprawdza dosłowne wzorce, liczby, zastrzeżenia	szybkie, tanie, dobrze audytowalne	słabe dla parafraz, semantyki i wielodokumentowości	niskie w prostych przypadkach, wysokie w parafrazach
Embedding similarity bez LLM	porównuje podobieństwo do treści źródłowych	dobrze jako retrieval / screening	nie daje werdyktu zgodności logicznej	średnie
Single-pass LLM judge	jeden prompt nad całym rekordem	szybki prototyp	bias judge’a, słaba audytowalność, mieszanie zadań	wysokie

Metoda	Co robi	Zalety	Główne wady	Ryzyko false positive
Extract → Retrieve → Verify	osobno claimy, dowody, werdykty	najwyższa kontrola i dobra zgodność z taskiem	więcej komponentów do strojenia	niskie do średnich
Extract → Retrieve → Verify + consensus + abstention	wariant konserwatywny z dwoma ścieżkami i HITL	najlepszy profil pod minimalizację FP	wyższy koszt i złożoność	najniższe

Tabela jest syntezą wyników z prac o claim extraction, evidence retrieval i biasach LLM judge'ów oraz z dokumentacji evals/guardrails. ¹⁷

Modele komercyjne

Dla wariantu API-first najpraktyczniejsze są modele, które łączą: długi kontekst, poprawne structured outputs, dobrą wielojęzyczność oraz rozsądne ceny batch/flex. Aktualna dokumentacja OpenAI wskazuje GPT-5.5 jako flagowy model do złożonego rozumowania z kontekstem 1M tokenów i Structured Outputs; Anthropic oferuje Sonnet 4.5/4.6 i Haiku 4.5 z batch discount oraz rekomenduje Structured Outputs do gwarantowanego JSON; Gemini 3.1 Pro ma 1M tokenów kontekstu, function calling, structured output i flex/batch pricing; Mistral Large 3 jest nietypowym przypadkiem, bo jest jednocześnie open-weight i dostępny w API z bardzo niską stawką za token. ¹⁸

Model	Najmocniejsze strony	Istotne ograniczenia	Rola rekomendowana
OpenAI GPT-5.5	1M kontekstu, structured outputs, silne rozumowanie, multilingual	najwyższy koszt wśród tu porównywanych opcji batch	drugi filtr dla najtrudniejszych lub najwyższego ryzyka
Claude Sonnet 4.5/4.6	dobra jakość tekstowa, mocna wielojęzyczność, batch, structured outputs	trzeba osobno sprawdzić zachowanie na polskich materiałach, bo oficjalna tabela nie raportuje pl-PL	główny weryfikator API przy dobrym balansie jakości i ceny
Claude Haiku 4.5	niska cena batch, dobra do screeningów	słabszy od Sonnet/Opus w trudnym wnioskowaniu	pierwszy filtr / triage
Gemini 3.1 Pro	1M kontekstu, function calling, structured output, flex/batch	w dokumentacji status preview / pre-GA dla części funkcji; wymaga własnej stabilizacyjnej ewaluacji	mocny drugi filtr, zwłaszcza dla długich dokumentów i PDF
Mistral Large 3	open-weight, 256k, function calling, structured outputs, bardzo niski koszt API	bardziej złożone self-hosting i obsługa MoE	drugi filtr w hybrydzie albo ekonomiczny weryfikator API

To porównanie opiera się na oficjalnych dokumentacjach modeli, cennikach i funkcji strukturalnych aktualnych na 1 czerwca 2026 r. ¹⁹

Modele open-weight i open-source

Jeżeli priorytetem jest prywatność, rezydencja danych lub niższy koszt jednostkowy przy większej liczbie rekordów, sensowny shortlist otwierają modele open-weight. Qwen3 wnosi hybrydowe tryby thinking/non-thinking, 119 języków i function calling; Gemma 4 ma natywny function calling, 128k lub 256k kontekstu zależnie od rozmiaru i jest projektowana pod wydajne lokalne wykonanie; Mistral Small 3.1 ma 24B parametrów, 128k kontekstu, prędkość rzędu 150 tokenów/s i licencję Apache 2.0; Llama 4 Scout daje skrajnie długi kontekst, ale przez architekturę MoE jest mniej „prostym” wyborem operacyjnym niż gęste modele 24B–32B. ²⁰

Model	Oficjalnie potwierdzone cechy	Wniosek praktyczny
Mistral Small 3.1 24B	24B, 128k kontekstu, ok. 150 tok/s, Apache 2.0	najlepszy kandydat na lokalny pierwszy filtr
Qwen3-32B / 235B-A22B	tryb hybrydowy, 119 języków, function calling, benchmarki szybkości i footprintu	bardzo mocna rodzina do lokalnego verify lub ensemble
Gemma 4 small/medium	small = 128k, medium = 256k, on-device focus, native function-calling	dobra opcja do prywatnych wdrożeń o średniej skali
Llama 4 Scout	17B aktywnych, 109B łącznie, do 10M kontekstu	interesujący przy ekstremalnie długich kontekstach, ale operacyjnie bardziej wymagający
Mistral Large 3	41B aktywnych, 675B łącznie, 256k, structured outputs, function calling	open-weight front-runner, ale do self-hostingu wymaga istotnie większego budżetu infrastrukturalnego

Podsumowanie ma charakter praktyczny, nie leaderboardowy: do pierwszego produkcyjnego pilota nie trzeba „największego” modelu; częściej lepsze efekty daje **mniejszy, tańszy model do claim extraction/screeningu i mocniejszy model tylko do drugiego filtra**. ²¹

Projekt promptów, wyjść i niepewności

Prompt powinien przypominać **instrukcję koderską**, nie zwykłe pytanie. Najlepiej sprawdzają się prompty, które wyraźnie rozdzielają rolę, codebook, dokumenty, claim, dopuszczalne etykiety i format odpowiedzi. Anthropic explicite rekomenduje strukturyzację promptów tagami XML, a OpenAI i Google podkreślają rolę precyzyjnych ograniczeń, realistycznych few-shot examples i iteracyjnej ewaluacji promptu na zestawie testowym. ²²

Dla tego zadania nie rekomenduję jawnego, rozwlekłego chain-of-thought przechowywanego w logach produkcyjnych. Klasyczne CoT i self-consistency potrafią poprawiać wyniki na zadaniach złożonego rozumowania, ale badania nad faithfulness pokazują, że zewnętrznie wygenerowany tok rozumowania nie zawsze wiernie odzwierciedla faktyczny proces decyzyjny modelu. W praktyce walidacji marketingowej lepiej zlecić modelowi „myślenie” wewnętrzne lub ograniczone, a zapisywać tylko:

etykiety, krótkie reason codes, evidence spans, ewentualnie zwięzły rationale summary. To daje lepszy audyt i mniejsze ryzyko złudnej interpretowalności. ²³

Few-shot warto stosować ostrożnie i zadaniowo. Oficjalne przewodniki OpenAI, Anthropic i Gemini potwierdzają, że dobrze dobrane przykłady poprawiają zgodność formatu i konsekwencję, ale OpenAI zarazem zaznacza, że dla silnych modeli reasoning warto zacząć od zero-shot, a few-shot dodać dopiero wtedy, gdy są złożone wymagania lub specyficzny styl odpowiedzi. W tym use case najlepsze są **krótkie, zróżnicowane przykłady graniczne**: poprawny claim liczbowy, utrata kwalifikatora, sprzeczność z datą, marketingowy superlative bez dowodu, przypadek niejednoznaczny oraz case do eskalacji. ²⁴

Poniżej przykład szablonu do ekstrakcji claimów:

```
<rola>
Jesteś konserwatywnym koderem walidacyjnym. Twoim zadaniem nie jest
"zgadywać", lecz wyodrębniać tylko takie claimy, które można zweryfikować
względem materiałów źródłowych.
</rola>

<codebook>
- Wyodrębniaj tylko claimy weryfikowalne.
- Rozbijaj zdania złożone na claimy atomowe.
- Jeśli sens zdania zależy od brakującego kontekstu, oznacz ambiguity=true.
- Nie oceniaj jeszcze zgodności; tylko ekstrakcja.
</codebook>

<etykiety_wyjsciowe>
Każdy claim ma pola:
claim_id, claim_text, claim_type, verifiable, ambiguity, missing_context,
priority
</etykiety_wyjsciowe>

<przyklad>
Wejście: "Krem redukuje widoczność zmarszczek nawet o 40% po 14 dniach."
Wyjście:
[
  {
    "claim_id": "c1",
    "claim_text": "Krem redukuje widoczność zmarszczek o 40% po 14 dniach.",
    "claim_type": "numerical_benefit",
    "verifiable": true,
    "ambiguity": false,
    "missing_context": null,
    "priority": "high"
  }
]
</przyklad>

<input>
{{candidate_text}}
</input>
```



```
<instrukcja>
Zwróć wyłącznie obiekt zgodny z JSON Schema.
</instrukcja>
```

Ten wzorzec wynika z połączenia praktyk prompt structuring z badaniami nad claim extraction i jakościowym kodowaniem. ²⁵

Szablon do właściwej weryfikacji powinien wymuszać **rozłączność etykiet** i dowody:

```
<rola>
Jesteś konserwatywnym weryfikatorem zgodności marketingowej.
</rola>

<task>
Oceń JEDEN claim wyłącznie na podstawie przekazanych evidence spans.
Jeśli dowód jest niewystarczający, nie zgaduj.
</task>

<allowed_labels>
SUPPORTED
CONTRADICTED
NOT_SUBSTANTIATED
QUALIFIER_LOSS
AMBIGUOUS
OUT_OF_SCOPE
</allowed_labels>

<decision_rules>
- SUPPORTED: claim jest jasno wsparty przez evidence spans.
- CONTRADICTED: evidence spans jasno przeczą claimowi.
- NOT_SUBSTANTIATED: claim nie jest wyraźnie wsparty ani obalony.
- QUALIFIER_LOSS: claim pomija istotny warunek, zakres, populację, czas lub zastrzeżenie obecne w źródle.
- AMBIGUOUS: sensem claimu lub dowodów nie da się zarządzać bez arbitralnej interpretacji.
</decision_rules>

<claim>
{{atomic_claim}}
</claim>

<evidence_spans>
{{top_k_spans}}
</evidence_spans>

<output_fields>
label
confidence_0_1
evidence_span_ids
```

```
reason_codes
short_rationale
requires_human_review
</output_fields>
```

<instrukcja>

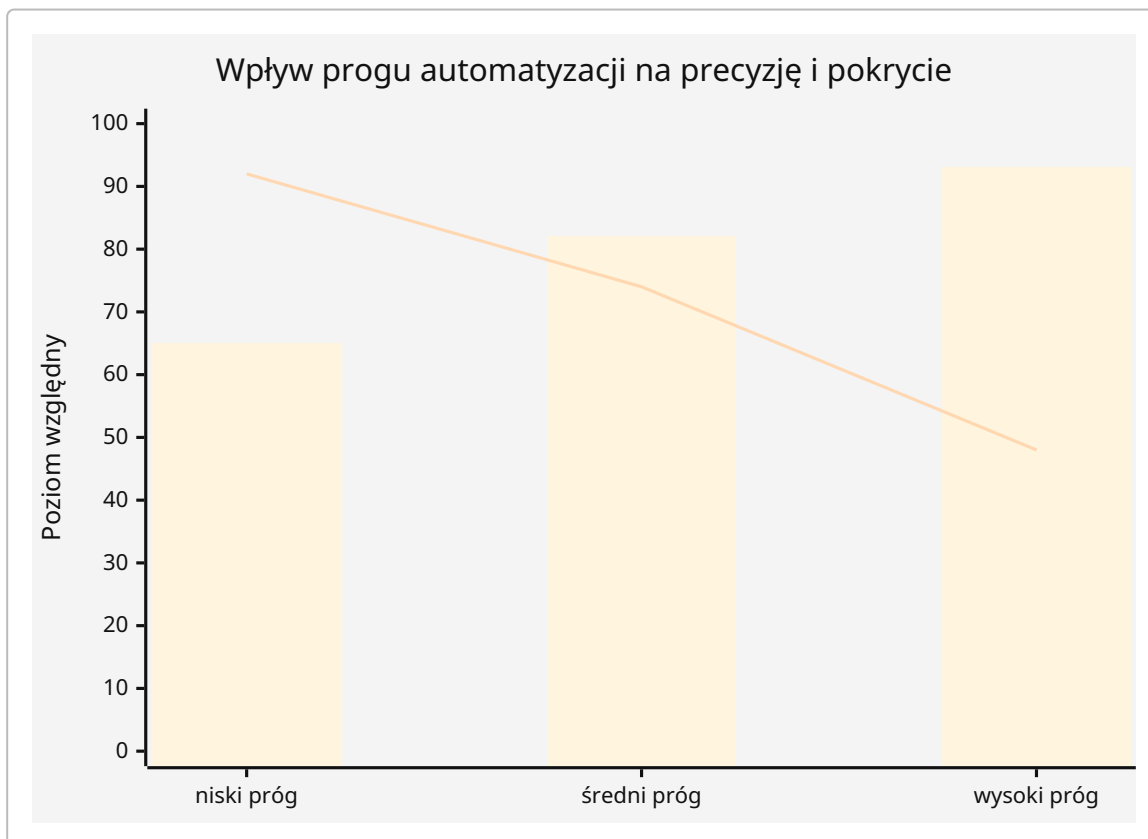
Myśl ostrożnie, ale zwróć wyłącznie wynik zgodny z JSON Schema.
Jeżeli confidence < {{threshold}} lub brak jednoznacznego dowodu, ustaw
requires_human_review=true.

</instrukcja>

Przy tym promptcie warto mieć 4–8 krótkich przykładów few-shot, ale tylko wtedy, gdy reprezentują rzeczywiste edge cases z projektu. Przykłady nie mogą przeczyć codebookowi, bo dokumentacje modelowe jasno ostrzegają, że niespójność między instrukcją a examples pogarsza wyniki. ²⁶

Najważniejsza warstwa po promptach to **kalibracja**. Sygnał `confidence_0_1` z modelu nie może być traktowany jak gotowe prawdopodobieństwo. Należy go skalibrować na zbiorze dev przez reliability diagrams i prosty kalibrator post-hoc, a następnie zamienić na politykę decyzji: niska pewność = review, średnia = drugi filtr, wysoka = automatyzacja. Survey o confidence estimation i prace o calibration pokazują, że właśnie ten krok przesądza, czy selective automation jest sensowna. ²⁷

Schematycznie kompromis między precyzją a pokryciem wygląda tak:



Wykres jest ilustracyjny, ale zasada jest realna: im bardziej konserwatywny próg automatycznego flagowania, tym wyższa precyzja i mniejsze ryzyko false positives, lecz tym większy wolumen rekordów trafi do człowieka. Ponieważ w marketingowej walidacji koszt biznesowy false positive bywa bardzo

wysoki, lepiej stroić system na **wysoką precyzję pozytywną klasy** niż na maksymalny F1 za wszelką cenę. ²⁸

Praktyczny workflow HITL powinien mieć trzy pasma decyzji. Pasma zielone: auto-pass przy wysokiej zgodności i wysokiej kalibracyjnej pewności. Pasma czerwone: auto-flag tylko po konsensusie dwóch ścieżek i zgodzie na poziomie claim + evidence. Pasma szare: review dla wszystkich przypadków z niską pewnością, sporami między modelami, claimami liczbowymi, comparatives, claims regulowanymi, konfliktem między dokumentami albo nową kategorią produktu. To jest bezpośrednie przełożenie wniosków z badań o abstention, biasie judge'ów i task-specific evals. ²⁹

Ewaluacja i protokół testowy

Benchmark lokalny powinien być ważniejszy niż publiczne leaderboards. Publiczne zbiory są potrzebne do sanity-checku komponentów, ale końcowa akceptacja powinna opierać się na **wewnętrznym gold secie** zbudowanym z realnych materiałów firmy, ponieważ to on odzwierciedla prawdziwy rozkład claimów, dokumentów i edge cases. Dokumentacje evals od OpenAI i Anthropic podkreślają, że ewaluacje mają odwzorowywać real-world task distribution i być wielowymiarowe, a nie tylko „sprawdzać benchmark”. ³⁰

Metryki i odpowiedniki zgodności międzykoderowej

Na poziomie binarnym lub wieloklasowym podstawowe metryki są standardowe: **precision = TP / (TP + FP)**, **recall = TP / (TP + FN)**, **F1** jako harmoniczna średnia precision i recall, oraz **false positive rate = FP / (FP + TN)**. W klasach niebalansowanych — a tak zwykle bywa, bo większość materiałów nie zawiera jawnych błędów — precision i FPR są ważniejsze niż sam F1. Kalibrację należy mierzyć reliability diagramem oraz błędem kalibracji; do raportowania warto dodać też coverage automatyzacji. ³¹

Dla analogów inter-annotator agreement najlepiej przyjąć trzy poziomy. **Human-human pre-adjudication** mierzy jakość codebooka i zrozumiałość zadania. **Human-LLM** mierzy, jak blisko model emuluje ludzkiego koder. **Multi-LLM / multi-prompt / multi-seed** mierzy stabilność samego systemu. Do par porównań używa się **Cohen's kappa**, a dla więcej niż dwóch „koderów”, różnych skal lub braków danych — **Krippendorff's alpha**. To jest szczególnie ważne, bo współczesne badania nad LLM annotators pokazują, że „procent zgodności” bywa mylący i że trzeba raportować chance-corrected agreement. ³²

Metryka	Pytanie, na które odpowiada	Jak używać w tym projekcie
Precision	Ile auto-flagów było trafnych?	główna metryka dla minimalizacji false positives
Recall	Ile realnych problemów wykryto?	ważna, ale wtórna wobec precision przy drogim FP
F1	Jaki jest kompromis precision/recall?	raportować, ale nie optymalizować ślepo
FPR	Ile poprawnych rekordów błędnie oznaczono?	metryka krytyczna dla biznesu
Calibration / reliability	Czy 0,8 pewności naprawdę oznacza ok. 80% trafności?	warunek selektywnej automatyzacji

Metryka	Pytanie, na które odpowiada	Jak używać w tym projekcie
Cohen's kappa	Jak LLM zgadza się z jednym koderem lub złotą adnotacją?	porównania pairwise
Krippendorff's alpha	Jaka jest zgodność wielu koderów lub wielu wykonanych ocen?	porównania human-human-human / human-LLM-prompt

To zestawienie opiera się na standardowych definicjach metryk klasyfikacyjnych oraz klasycznej literaturze agreement metrics. ³³

Protokół testowy

Rekomendowany protokół ma sześć kroków. Najpierw należy zbudować złoty zbiór z rozkładem odpowiadającym produkcji: według kanału, języka, typu claimu, brandu, kategorii produktu i risk-tier. Następnie trzeba zamrozić wersje dokumentów i promptów, aby uniknąć „poruszającego się celu”. Później ocenia się osobno komponenty: claim extraction, retrieval, claim verification i record aggregation. Dopiero potem stroi się progi na dev secie i wykonuje jeden ślepy test końcowy na teście odłożonym przez kampanię, produkt i czas, a nie przez losowy rekord. Na końcu robi się slice analysis i rekaliibrację, ale bez przepalania testu na kolejne iteracje. ³⁴

Istotne jest, aby split nie dopuszczał łatwego przecieku. Ten sam produkt, ten sam claim source lub blisko parafrazowane kampanie nie powinny występować i w dev, i w test. W przeciwnym razie wynik mierzy pamięć o wzorcu, a nie zdolność walidacji. Ryzyko takiej kontaminacji benchmarków jest dobrze znane także w szerszej literaturze evals. ³⁵

Istotność statystyczna

Dla porównań konfiguracji model-prompt-threshold rekomenduję raportować **różnice metryk wraz z 95% paired bootstrap confidence intervals**. W NLP to praktyczna i szeroko rekomendowana metoda, bo działa dla niemal dowolnej metryki, w tym precision, recall, F1 czy kappa, i kieruje uwagę na wielkość efektu, nie tylko na etykietę „significant / not significant”. Dla par binarnych decyzji na tych samych rekordach — na przykład „czy model A auto-flaguje inaczej niż model B” — stosuje się **McNemar test**. ³⁶

Ponieważ dane marketingowe zwykle są klastrowane — rekordy należą do kampanii, produktów i wersji materiałów — bootstrap najlepiej liczyć blokowo po kampaniach lub produktach, a nie po pojedynczych rekordach, jeśli zależności wewnątrz klastra są silne. To jest rekomendacja inferencyjna wynikająca z ogólnej logiki bootstrapu i konstrukcji evals; bez tego łatwo zaniżyć niepewność porównań. ³⁷

Analiza błędów i testy odporności

Analiza błędów nie może kończyć się na confusion matrix. Trzeba tagować pomyłki co najmniej do jednej z sześciu przyczyn źródłowych: **retrieval miss, zła dekompozycja claimu, utrata kwalifikatora, błąd liczbowy/jednostkowy, zła agregacja dokumentów, nieadekwatna polityka progów lub kalibracji**. Taki rozkład błędów szybko pokazuje, czy model trzeba zmienić, czy raczej poprawić preprocessing, retrieval lub codebook. ³⁸

Testy odporności powinny zawierać zestaw sztucznych, ale realistycznych perturbacji: zamiana encji, zmiana liczby z 30% na 33%, usunięcie „dla wybranych klientów”, negacja, mylenie zakresu czasu, przeniesienie ważnego zastrzeżenia do przypisu, konflikty między dwiema wersjami źródła, długie

nieistotne appendixy, wielojęzyczne parafrazy i prompt injection zaszyty w dokumencie źródłowym. Literatura o biasach judge'ów, claim verification i faithfulness CoT daje mocne uzasadnienie dla takiego stress-testingu. ³⁹

Zbiory do benchmarkingu

Publiczne zbiory warto podzielić na te, które testują **claim extraction**, i te, które testują **verification against evidence**. Żaden z nich nie jest w pełni marketingowy, ale razem dobrze pokrywają docelowy pipeline. ⁴⁰

Zbiór	Do czego służy najlepiej	Dlaczego jest użyteczny tutaj
Wewnętrzny gold set	końcowa akceptacja systemu	jedyny zbiór odzwierciedlający realny rozkład materiałów i polityk firmy
FEVER	podstawowa claim verification z dowodami	dobry do sanity-checku Supported / Refuted / NEI i evidence retrieval ⁴¹
SciFact	claim verification + rationales	przydatny dla rygorystycznego evidence-based reasoning i claimów eksperckich ⁴²
AVeriTeC	real-world web claim verification z Q/A i justifications	najbliższy realistycznym, złożonym claimom i pracy na wielu dowodach; ma 4568 claimów i κ=0,619 na verdictach ⁴³
DocNLI	document-level entailment	dobry do ćwiczenia wnioskowania nad całymi dokumentami ⁴⁴
ContractNLI	real-world document NLI	dobry odpowiednik pracy z dokumentami formalnymi i warunkami / zastrzeżeniami ⁴⁵
FEVERFact / Claimify-style benchmarks	claim extraction	pomagają mierzyć atomowość, coverage i decontextualization ekstrakcji claimów ⁴⁶

Kolejność priorytetów powinna być taka: najpierw wewnętrzny benchmark; potem publiczne zbiory do testu komponentów; nigdy odwrotnie. Publiczne wyniki są użyteczne diagnostycznie, ale nie są kryterium gotowości wdrożeniowej dla walidacji marketingu. ⁴⁷

Koszty, harmonogram i ryzyka

Szacunki kosztowe trzeba liczyć nie „na 1 prompt”, tylko na cały pipeline. Dla porównania przyjmuję wykonalny scenariusz: **10 000 rekordów**, po preprocessingu średnio **1500 tokenów wejściowych i 250 wyjściowych** dla pierwszego filtra oraz **20% rekordów** przepuszczanych do drugiego filtra z **2000 tokenami wejściowymi i 300 wyjściowymi**. To daje łącznie około **19 mln tokenów wejściowych i 3,1 mln tokenów wyjściowych** w pełnym pipeline. Ceny poniżej opierają się na oficjalnych cennikach dostępnych 1 czerwca 2026 r.; realny koszt zależy od długości dokumentów, cache, batch/flex i odsetka przypadków eskalowanych. ⁴⁸

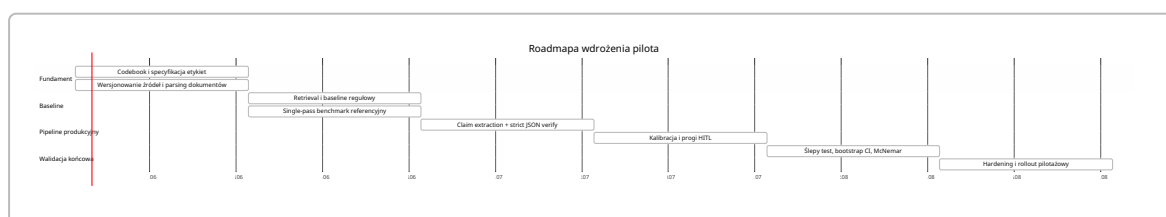
Wariant	Założenie modelowe	Szacunkowy koszt dla 10k rekordów
Jeden mocny model batch	GPT-5.5 Batch dla obu warstw	ok. 94 USD
Jeden mocny model batch	Claude Sonnet 4.5 Batch dla obu warstw	ok. 51,8 USD
Jeden mocny model flex/batch	Gemini 3.1 Pro Flex/Batch dla obu warstw	ok. 37,6 USD
Jeden ekonomiczny model API	Mistral Large 3 dla obu warstw	ok. 14,2 USD
Kaskada konserwatywna	Haiku 4.5 Batch jako screening + Sonnet 4.5 Batch jako drugi filtr	ok. 24,3 USD
Hybryda prywatna	lokalny Mistral Small 3.1 / Qwen3-32B + Mistral Large 3 API tylko dla 20% trudnych przypadków	ok. 2,9 USD kosztu API + koszt GPU

W tablicy widać najważniejszy wniosek praktyczny: **najtańszy nie musi być najgorszy, jeśli służy tylko do screeningu**, a największą oszczędność daje uzależnienie drogiej weryfikacji od pasma niepewności. To jest jeden z najsilniejszych argumentów za selektywną automatyzacją zamiast jednego modelu dla wszystkiego. ⁴⁹

Dla self-hostingu sensowne są przede wszystkim gęste modele średniej wielkości. Mistral Small 3.1 ma 24B parametrów, więc **szacunkowo** same wagi mieszczą się w okolicach ~48 GB w bf16, ~24 GB w 8-bit i ~12 GB w 4-bit, bez uwzględnienia KV cache i narzutu serwera; Qwen3-32B jest odpowiednio cięższy. Ponieważ H100 ma 80 GB lub 94 GB pamięci, a A100 występuje w wariacie 80 GB, te klasy modeli są operacyjnie realne na pojedynczym GPU klasy data center, podczas gdy Mistral Large 3 jako 675B MoE to już inna liga operacyjna. To jest celowo ostrożny, porządkowy szacunek wynikający z liczby parametrów i oficjalnych specyfikacji GPU. ⁵⁰

Jeżeli pierwszy filtr zrealizować lokalnie na Mistral Small 3.1, oficjalny komunikat Mistrala mówi o prędkości rzędu **150 tokenów/s**. To oznacza, że przebieg rzędu kilkunastu–kilkudziesięciu milionów tokenów jest wykonalny w skali jednego–kilku dni GPU czasu na pojedynczej karcie, a przy równolegleniu i batchingu szybciej. W praktyce więc lokalny screening i API-only second pass są dziś rozwiązaniem w pełni wykonalnym, a nie eksperymentalnym. ⁵¹

Roadmapa wdrożeniowa może wyglądać tak:



W pierwszych dwóch tygodniach nie należy jeszcze „walczyć o leaderboard promptów”; największy zwrot z pracy dają wtedy codebook, wersjonowanie źródeł i dobrze zbudowany gold set. Dopiero potem opłaca się stroić modele i progi. To wynika zarówno z praktyki badań jakościowych, jak i z oficjalnych przewodników evals. ⁵²

Najważniejsze ryzyka i ich realne mitigacje są następujące:

Ryzyko	Objaw	Mitigacja
Retrieval miss	model twierdzi „brak dowodu”, choć dowód istnieje	osobna ocena retrieval recall, hybrydowy retrieval, gold spans na dev secie
Bias judge’a	różne werdykty po zmianie kolejności lub formatu	order swap, dwa niezależne przebiegi, prompt variance audit
Overconfidence	model rzadko mówi „nie wiem”, ale myli się pewnie	kalibracja, abstention, reward dla właściwej niepewności
Utrata kwalifikatora	model zbyt często ocenia parafrazę jako sprzeczność	osobna etykieta QUALIFIER_LOSS i przykłady few-shot z zastrzeżeniami
Dryf źródeł i wersji	ten sam claim jest raz dobry, raz zły, bo użyto innej wersji źródła	wersjonowanie dokumentów i zamrożone snapshoty
Nadmierny koszt	zbyt dużo rekordów trafia do mocnego modelu lub do ludzi	kaskada modeli, batch/flex, risk-tiering, okresowa rekalkulacja progów
Ułuda „pełnej automatyzacji”	dobrze średnie metryki maskują kosztowne edge cases	slice analysis, ślepy test końcowy, obowiązkowy pilot z człowiekiem

Każde z tych ryzyk jest już dobrze opisane przez literaturę o claim verification, LLM-as-a-judge, faithfulness CoT i selective prediction; najskuteczniejsze środki zaradcze nie są spekulatywne, tylko dobrze uzasadnione empirycznie. ⁵³

Końcowa rekomendacja jest więc prosta: **wdrażać tylko pipeline etapowy, evidence-grounded, z koderskim codebookiem, structured outputs, kalibracją, abstention i obowiązkowym human review dla szarej strefy**. Jeśli projekt ma być skuteczny i obronny metodologicznie, „najpierw benchmark i polityka decyzji, potem model” jest lepszą ścieżką niż „najpierw największy model, potem zobaczmy”. ⁵⁴

¹ ⁶ ¹⁴ ⁵² <https://journals.sagepub.com/doi/10.1177/00491241251339188>
<https://journals.sagepub.com/doi/10.1177/00491241251339188>

² <https://openai.com/index/why-language-models-hallucinate/>
<https://openai.com/index/why-language-models-hallucinate/>

³ <https://eur-lex.europa.eu/LexUriServ/LexUriServ.do?uri=OJ%3AL%3A2005%3A149%3A0022%3A0039%3Aen%3APDF>
<https://eur-lex.europa.eu/LexUriServ/LexUriServ.do?uri=OJ%3AL%3A2005%3A149%3A0022%3A0039%3Aen%3APDF>

⁴ <https://developers.openai.com/api/docs/guides/structured-outputs>
<https://developers.openai.com/api/docs/guides/structured-outputs>

⁵ ¹⁷ ³⁸ ⁴⁰ <https://aclanthology.org/2025.acl-long.348/>
<https://aclanthology.org/2025.acl-long.348/>

⁷ ¹³ ¹⁶ ⁵³ <https://aclanthology.org/2024.naacl-long.196/>
<https://aclanthology.org/2024.naacl-long.196/>

- 8 34 54 <https://platform.claude.com/docs/en/test-and-evaluate/develop-tests>
<https://platform.claude.com/docs/en/test-and-evaluate/develop-tests>
- 9 41 <https://aclanthology.org/N18-1074/>
<https://aclanthology.org/N18-1074/>
- 10 44 <https://aclanthology.org/2021.findings-acl.435/>
<https://aclanthology.org/2021.findings-acl.435/>
- 11 12 <https://fever.ai/dataset/averitec.html>
<https://fever.ai/dataset/averitec.html>
- 15 <https://aclanthology.org/2024.acl-long.511/>
<https://aclanthology.org/2024.acl-long.511/>
- 18 19 <https://developers.openai.com/api/docs/models>
<https://developers.openai.com/api/docs/models>
- 20 https://qwen.readthedocs.io/en/latest/getting_started/concepts.html
https://qwen.readthedocs.io/en/latest/getting_started/concepts.html
- 21 51 <https://mistral.ai/news/mistral-small-3-1/>
<https://mistral.ai/news/mistral-small-3-1/>
- 22 24 25 <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/multishot-prompting>
<https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/multishot-prompting>
- 23 <https://proceedings.neurips.cc/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html>
<https://proceedings.neurips.cc/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html>
- 26 <https://developers.openai.com/api/docs/guides/prompt-engineering>
<https://developers.openai.com/api/docs/guides/prompt-engineering>
- 27 https://aclanthology.org/2024.naacl-long.366.pdf?utm_source=chatgpt.com
https://aclanthology.org/2024.naacl-long.366.pdf?utm_source=chatgpt.com
- 28 https://scikit-learn.org/stable/auto_examples/model_selection/plot_precision_recall.html
https://scikit-learn.org/stable/auto_examples/model_selection/plot_precision_recall.html
- 29 49 <https://openreview.net/pdf?id=1DIIdt2YOPw>
<https://openreview.net/pdf?id=1DIIdt2YOPw>
- 30 <https://developers.openai.com/api/docs/guides/evals>
<https://developers.openai.com/api/docs/guides/evals>
- 31 33 https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_recall_fscore_support.html
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_recall_fscore_support.html
- 32 <https://journals.sagepub.com/doi/10.1177/001316446002000104>
<https://journals.sagepub.com/doi/10.1177/001316446002000104>
- 35 <https://anthropic.com/engineering/eval-awareness-browsecomp>
<https://anthropic.com/engineering/eval-awareness-browsecomp>
- 36 37 <https://aclanthology.org/2022.lrec-1.640.pdf>
<https://aclanthology.org/2022.lrec-1.640.pdf>

- 39 <https://aclanthology.org/2025.ijcnlp-long.18/>
<https://aclanthology.org/2025.ijcnlp-long.18/>
- 42 <https://aclanthology.org/2020.emnlp-main.609/>
<https://aclanthology.org/2020.emnlp-main.609/>
- 43 <https://openreview.net/forum?id=fKzSz0oyaI>
<https://openreview.net/forum?id=fKzSz0oyaI>
- 45 <https://aclanthology.org/2021.findings-emnlp.164/>
<https://aclanthology.org/2021.findings-emnlp.164/>
- 46 <https://arxiv.org/abs/2502.04955>
<https://arxiv.org/abs/2502.04955>
- 47 <https://developers.openai.com/api/docs/guides/evaluation-best-practices>
<https://developers.openai.com/api/docs/guides/evaluation-best-practices>
- 48 <https://openai.com/api/pricing/>
<https://openai.com/api/pricing/>
- 50 <https://www.nvidia.com/en-us/data-center/h100/>
<https://www.nvidia.com/en-us/data-center/h100/>